

Programación Reactiva

Sistemas que reaccionan a eventos y cambios

PARADIGMAS DE PROGRAMACIÓN

UNIVERSIDAD

¿Qué es Programación Reactiva?

→ Definición

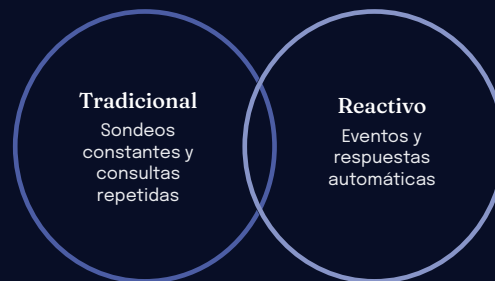
La programación reactiva es un paradigma de programación **asíncrona** **orientado al flujo de datos** y a la **propagación del cambio**.

→ Explicación simple

Un paradigma donde el programa **reacciona automáticamente** a cambios y eventos, en lugar de preguntar constantemente si algo cambió.

→ La idea de "reaccionar"

El código se **suscribe** a fuentes de datos y actúa automáticamente cuando llega nueva información.



i **🔔 Analogía del timbre:** No estás mirando la puerta cada segundo – esperas a que suene el timbre y *entonces* reaccionas.



¿Por qué surge la Programación Reactiva?

Problemas modernos

Las aplicaciones actuales manejan **millones de usuarios simultáneos**, datos en tiempo real y alta concurrencia que los modelos clásicos no soportan bien.

Tiempo real

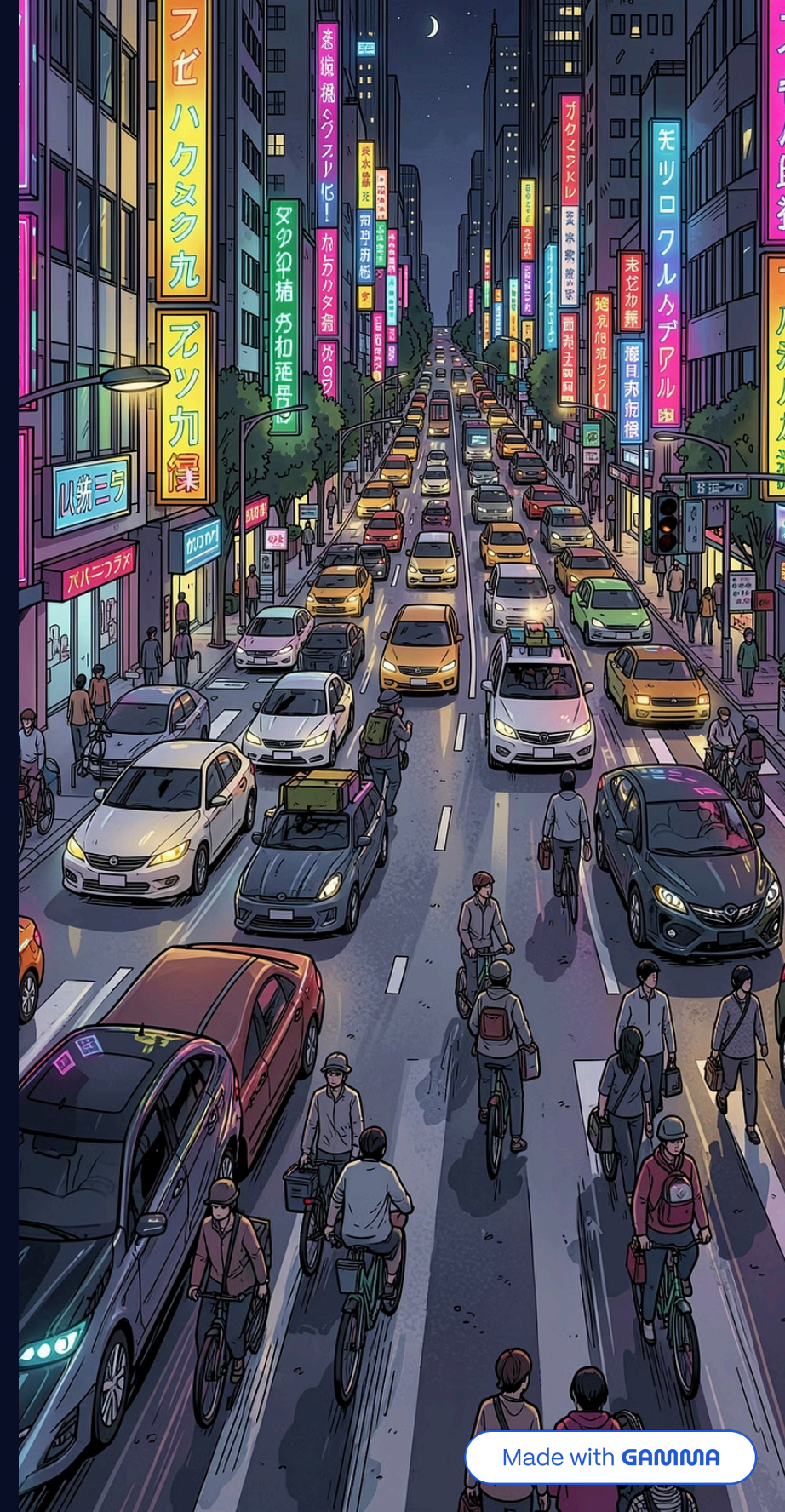
Los usuarios esperan **respuestas instantáneas**: notificaciones, actualizaciones de precio, mensajes – todo sin recargar la página.

Muchas interacciones simultáneas

Un solo servidor puede recibir **miles de eventos por segundo**: clics, peticiones API, sensores IoT, streams de vídeo.

Ejemplos

Youtube gestiona streams de vídeo · **Uber** actualiza posiciones en vivo · **WhatsApp** entrega mensajes al instante



La programación reactiva es un paradigma de programación **asíncrona orientado al flujo de datos** y a la **propagación del cambio**.

Streams y Eventos

¿Qué es un stream?

Un **stream** es una secuencia de eventos ordenados en el tiempo. Puede emitir tres tipos de señales:

Ejemplos reales de streams



Clics del usuario

Cada clic es un evento en el stream de interacciones de la interfaz.



Mensajes de chat

Cada mensaje recibido es un evento que fluye por el stream de comunicación.



Sensores y APIs

Lecturas de temperatura, GPS o respuestas HTTP forman streams continuos.



Observable, Observer y Subscriber



1

Observable

Es la **fuentes de datos**. Emite eventos a lo largo del tiempo. Puede ser finito (una petición HTTP) o infinito (clics del ratón).

2

Observer

Define **qué hacer** con cada evento emitido: cómo manejar un valor, un error o la finalización del stream.

3

Subscriber

Se suscribe al Observable para empezar a recibir eventos. Sin suscriptor, el Observable no emite nada.



Analogía YouTube: El canal es el **Observable**, tú eres el **Subscriber**, y la notificación de campana es el mecanismo de **Observer**.

Estrategias de Propagación del Cambio

Las aplicaciones reactivas manejan la difusión de eventos y datos a través de diferentes mecanismos.



Push (Empuje)

La fuente de datos **envía activamente** los eventos a los suscriptores tan pronto como están disponibles. Los suscriptores simplemente escuchan y reaccionan.



Pull (Tirón)

El suscriptor es quien **solicita** los datos a la fuente cuando los necesita. La fuente responde solo cuando se le pide, sin iniciativa propia.



Push-Pull (Híbrido)

Una combinación donde la fuente de datos envía una notificación de cambio. Sin embargo, la notificación no contiene toda la información necesaria por lo que el consumidor debe consultar la fuente para obtener más información (el valor concreto) tras recibir la notificación; esta es la parte de «pull».

Características de una Aplicación Reactiva

Responsivos

Responden rápidamente a cualquier entrada del usuario o evento del sistema

Elásticos

Se adaptan dinámicamente a cambios en la carga de trabajo, escalando recursos según sea necesario

Orientados a Mensajes

Comunican mediante paso de mensajes asíncrono entre componentes desacoplados

Resilientes

Mantienen la funcionalidad incluso ante fallos, con recuperación automática

switchMap, debounce y throttle



switchMap

Cancela la petición anterior y lanza una nueva cuando llega un evento. **Ideal para búsquedas en tiempo real**: si el usuario sigue escribiendo, descarta la búsqueda anterior y solo procesa la última.



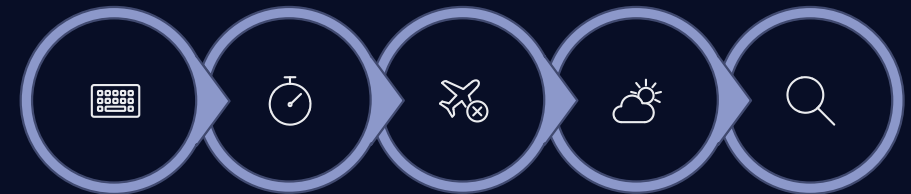
debounce

Espera a que el usuario **deje de emitir eventos** durante X milisegundos antes de actuar. Evita llamadas API por cada tecla pulsada.



throttle

Limita la frecuencia de eventos a **uno por intervalo de tiempo**, ignorando los intermedios. Útil para scroll o resize.



Escribir

Debounce
300ms

switchMap
cancela

Nueva
petición

Mostrar
resultados

✓  **Ejemplo Google Search:** Sin debounce + switchMap, cada letra dispararía una petición al servidor. Con ellos, solo se lanza una petición cuando el usuario termina de escribir.

Backpressure

El problema

Ocurre cuando el **productor emite datos más rápido** de lo que el consumidor puede procesarlos. Es uno de los desafíos más críticos en sistemas reactivos.

Estrategias de solución

→ Buffer

Almacena temporalmente los eventos en una cola con límite máximo.

→ Drop

Descarta eventos cuando el consumidor está ocupado, priorizando los más recientes.

→ Throttle del productor

El consumidor señala al productor que reduzca su velocidad de emisión.

⚠️ 📚 **Analogía tareas acumuladas:** Es como recibir 100 correos urgentes mientras solo puedes responder 5 por hora – la bandeja de entrada se desborda y pierdes el control.

⚠️ Saturación

La cola de eventos se desborda y el sistema colapsa.

💾 Memoria

Los eventos sin procesar consumen RAM hasta agotar los recursos.

🐢 Lentitud

El sistema se degrada progresivamente hasta volverse inutilizable.

Lenguajes de programación

ReactiveX



ReactiveX es una implementación de programación reactiva que permite trabajar con flujos de datos síncronos y asíncronos mediante observables y operadores reactivos.

Lenguajes compatibles con Reactivex

Lenguaje	Librería
Java	RxJava
JavaScript	RxJS
Python	RxPY (ejemplo)
Kotlin	RxKotlin
C#	Rx.NET
Swift	RxSwift
Dart	RxDart

Frameworks y plataformas

Spring WebFlux



Spring Framework 5 incorpora Spring WebFlux, permitiendo construir aplicaciones web reactivas y no bloqueantes.

Svelte



Svelte utiliza un enfoque reactivo para actualizar automáticamente la interfaz de usuario cuando cambian los datos.

Aplicaciones de la Programación Reactiva

Sistemas Distribuidos

Agregación asíncrona y resiliencia en microservicios.



Alta Concurrencia

Comunicación bidireccional masiva en tiempo real.



Interfaces Dinámicas (UI)

Flujos de eventos del usuario e interfaces reactivas.



IoT y Telemetría

Ingesta y filtrado de flujos de datos infinitos en movimiento.



1.Sistemas Distribuidos

En entornos de microservicios, la programación reactiva es esencial para la **agregación no bloqueante** y la gestión eficiente de la **contrapresión (backpressure)**.

Netflix

Orquestación de llamadas asíncronas concurrentes mediante streams con RxJava. Evita el bloqueo de hilos, asegurando una alta disponibilidad y baja latencia incluso con millones de usuarios.

Spotify

Composición de flujos diferidos y operadores de *fallback* para evitar caídas en cascada si un servicio externo falla, manteniendo la experiencia de usuario fluida.

The Netflix logo, consisting of the word "NETFLIX" in a bold, red, sans-serif font, set against a black rectangular background.

  **Concepto clave:** La **Backpressure** es crítica aquí. Es el mecanismo para que los sistemas que procesan datos avisen a los que los producen que reduzcan la velocidad, evitando la saturación y colapsos.

2. Interfaces Dinámicas

Secuencias de Eventos Temporales (Observables) en el Frontend

UBER

App Móvil

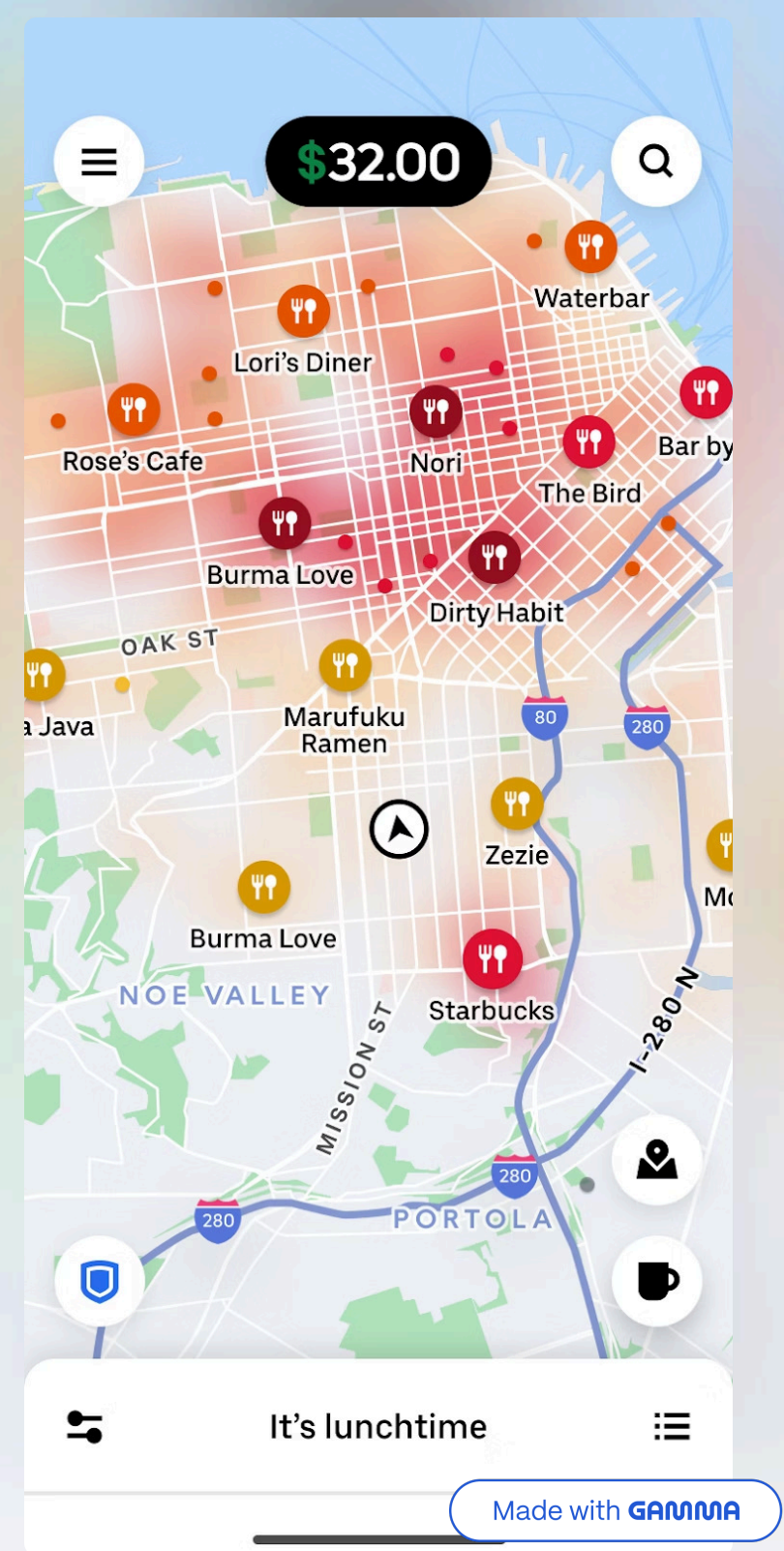
Coordenadas de **GPS** y datos de **tráfico** son tratados como flujos síncronos/asíncronos unificados. Permite un re-renderizado aislado y eficiente de componentes.

AIRBNB

Mapas Interactivos

Control del **Callback Hell** mediante operadores reactivos. Uso de `debounceTime` y `switchMap` para optimizar peticiones HTTP del usuario.

💡 Concepto Técnico Clave: Los **flujos del lado del cliente** (Client-side reactive streams) permiten manejar interacciones complejas de usuario de forma declarativa y asíncrona, mejorando la experiencia y el rendimiento.



3. Alta Concurrency

Conexiones Persistentes y Modelo Publish-Subscribe.

Concepto Técnico Clave: La **Arquitectura Dirigida por Eventos** elimina el *polling* constante, reduciendo la latencia y la carga del servidor para una experiencia más fluida.

DISCORD

SLACK

WebSockets

Conexiones de canal abierto **full-duplex** para un flujo continuo. Empuje (push) de eventos de estado concurrentes, optimizando el **Event Loop**.

Notificaciones

Arquitectura **Publish-Subscribe** dirigida por eventos. Los usuarios actúan como **Observers** de flujos centralizados, recibiendo actualizaciones instantáneas.

4. IoT y Telemetría

La programación reactiva permite gestionar el flujo constante de datos generados por millones de dispositivos, desde vehículos hasta sensores agrícolas.



Tesla

Ingesta masiva y procesamiento de telemetría en tiempo real antes del almacenamiento físico. Crucial para actualizaciones de software, análisis predictivo y optimización del rendimiento del vehículo.



John Deere

Filtros lógicos sobre streams infinitos bajo conectividad intermitente para automatización agrícola. Permite a la maquinaria tomar decisiones en campo de forma autónoma.



Concepto Técnico Clave: El Procesamiento de flujos de datos en movimiento (Stream Processing) es fundamental para extraer valor de datos que nunca "terminan", como los generados por dispositivos IoT.

Ventajas y Desventajas

Un análisis técnico comparativo para comprender la idoneidad de la Programación Reactiva en distintos contextos arquitectónicos.

Tipo	Ventaja / Desventaja	Explicación técnica	Por qué ocurre y qué resuelve / genera
✓ Ventaja	Mejor manejo de asincronía	Permite gestionar tareas de larga duración liberando hilos de ejecución mediante la composición declarativa de Observables.	Ocurre por la abstracción de secuencias de eventos y el uso eficiente de hilos (Schedulers). Resuelve la complejidad
✓ Ventaja	Mayor escalabilidad en sistemas concurrentes	Al operar de forma no bloqueante y orientada a eventos, un número reducido de hilos puede manejar una elevada concurrencia de solicitudes.	Evita la asignación de un hilo por petición y resuelve los cuellos de botella.
✓ Ventaja	Mejor respuesta ante eventos en tiempo real	Facilita arquitecturas "push-based", donde los datos se entregan a los consumidores tan pronto como están disponibles.	Resuelve la necesidad de actualizaciones instantáneas en aplicaciones.
✓ Ventaja	Manejo eficiente de flujos de datos	Ofrece un amplio conjunto de operadores para transformar, combinar, filtrar y reaccionar a secuencias de datos (streams) de manera declarativa y funcional.	Proporciona herramientas potentes para procesar datos continuos (ej. telemetría IoT, logs de servidores) de forma estructurada. Resuelve la complejidad de manipular grandes volúmenes de eventos con lógica de negocio compleja, mejorando la expresividad y reduciendo errores.
✓ Ventaja	Soporte para backpressure y control de carga	Integra mecanismos para que el consumidor pueda señalar al productor que disminuya la velocidad de emisión de datos si se encuentra saturado, previniendo el desbordamiento.	Es intrínseco al Reactive Streams Specification. Resuelve el problema de la sobrecarga del consumidor cuando el productor es más rápido, evitando fallos en cascada y manteniendo la estabilidad en sistemas distribuidos y de streaming de datos.
✗ Desventaja	Mayor curva de aprendizaje	Requiere un cambio de mentalidad desde el paradigma imperativo tradicional hacia un estilo de programación declarativo y funcional con Observables, Suscriptores y operadores.	Ocurre por la abstracción sobre el control de flujo y la gestión de estados. Genera una barrera inicial para desarrolladores no familiarizados con estos patrones, lo que puede ralentizar el desarrollo y aumentar la necesidad de capacitación.
✗ Desventaja	Debugging más complejo	La naturaleza asíncrona y la transformación de flujos de datos a través de múltiples operadores pueden dificultar el seguimiento del estado y la identificación de la causa raíz de los errores.	Se debe a la falta de un flujo de ejecución lineal y a la propagación de errores a través de Observables. Genera desafíos para usar depuradores tradicionales y requiere herramientas y técnicas específicas (ej. operadores de logging, diagramas de mármol) para diagnosticar problemas.
✗ Desventaja	Código más difícil de leer si no se estructura bien	Aunque declarativa, una cadena de operadores reactivos larga o mal organizada puede volverse intrincada y menos legible que su equivalente imperativo si no se aplican buenas prácticas.	Deriva de la flexibilidad y el poder de composición. Genera código espagueti si no se modulan y factorizan los flujos de datos, dificultando la comprensión para nuevos miembros del equipo o el mantenimiento a largo plazo.
✗ Desventaja	Mayor complejidad en el manejo de errores	La gestión de errores en Observables requiere el uso de operadores específicos (ej. <code>catchError</code> , <code>retry</code>) que deben ser aplicados cuidadosamente en el flujo, a diferencia de los bloques try-catch tradicionales.	Se debe a la naturaleza de la propagación de errores a través de un flujo. Si no se manejan explícitamente, los errores pueden terminar la suscripción del Observable y no ser detectados, generando un comportamiento inesperado o interrupciones en la aplicación.
✗ Desventaja	Sobrecosto innecesario en aplicaciones simples	La introducción de una librería reactiva (ej. RxJava, Reactor) y sus abstracciones puede ser una sobreingeniería para aplicaciones con pocas operaciones asíncronas o baja concurrencia.	Ocurre porque el beneficio de la programación reactiva se maximiza en escenarios complejos y con altos volúmenes de eventos. Genera una complejidad adicional sin un beneficio proporcional en el rendimiento o la mantenibilidad en aplicaciones CRUD sencillas o scripts básicos.

La elección entre programación reactiva e imperativa debe basarse en las necesidades del proyecto, la complejidad de la asincronía y la experiencia del equipo.

Ejemplo Práctico: Un Endpoint Reactivo

Este fragmento de código Java ilustra la construcción de un endpoint reactivo con Spring WebFlux, demostrando la aplicación de operadores reactivos para transformar y gestionar flujos de datos de forma asíncrona y no bloqueante.

```
@GetMapping("/notificaciones")
public Flux<Notificacion> obtenerNotificaciones() {
    return eventoService.obtenerEventos()
        .filter(evento -> evento.esImportante())
        .map(evento -> new Notificacion(evento.getUsuario(), evento.getMensaje()))
        .onErrorResume(error -> Flux.empty());
}
```



eventoService.obtenerEventos()

Inicia un **flujo de eventos** (Flux), que puede emitir cero, uno o múltiples ítems de forma asíncrona a lo largo del tiempo.



.filter(...)

Aplica una condición para **filtrar** solo los eventos que cumplen ciertos criterios, en este caso, solo los eventos `esImportante()`.



.map(...)

Transforma cada evento del flujo en un nuevo objeto, convirtiendo un `Evento` en una `Notificacion`.



.onErrorResume(...)

Define una estrategia de **recuperación de errores**. Si ocurre un fallo en el flujo, se retorna un `Flux` vacío, evitando que la aplicación falle.

  **Conclusión:** Este código encapsula los principios reactivos de forma concisa, permitiendo un manejo robusto y eficiente de las notificaciones, donde cada operación se realiza de manera no bloqueante.

Ejemplo Práctico: Flujo de Mensajes Reactivo con RxJS

Este fragmento de código JavaScript ilustra cómo utilizar la programación reactiva con RxJS para manejar, transformar y procesar un flujo de mensajes provenientes de un WebSocket, demostrando la capacidad de RxJS para orquestar eventos asíncronos de forma declarativa.

```
const mensajes$ = fromEvent(socket, "message");

mensajes$
  .pipe(
    map((msg: any) => JSON.parse(msg)),
    filter(msg => msg.texto.length > 0),
    map(msg => ({
      usuario: msg.usuario,
      texto: msg.texto.trim(),
      fecha: new Date()
    }))
  )
  .subscribe(msg => enviarATodos(msg));
```



`fromEvent(socket, "message")`

Crea un **Observable** a partir de los eventos "message" de un objeto WebSocket, estableciendo un flujo continuo de datos.



`.pipe(map(msg => JSON.parse(msg)))`

Utiliza el operador `map` para **deserializar** cada mensaje recibido (en formato JSON) a un objeto JavaScript.



`.filter(msg => msg.texto.length > 0)`

Aplica el operador `filter` para **omitir** los mensajes cuyo campo `texto` esté vacío, asegurando que solo se procesen mensajes válidos.



`.map(msg => ({...}))`

Transforma la estructura del objeto mensaje, añadiendo una marca de tiempo (`fecha`) y asegurando el formato deseado para el usuario y el texto.



`.subscribe(msg => enviarATodos(msg))`

El **suscriptor** final recibe los mensajes ya procesados y filtrados, y los distribuye a todos los clientes conectados a través de la función `enviarATodos`.



💡 Conclusión: Este pipeline de RxJS gestiona de forma elegante y eficiente el ciclo de vida de los mensajes, desde su recepción hasta su distribución, con una clara separación de responsabilidades en cada paso.

Ejemplo Práctico: Procesamiento de Logs Reactivo

Este ejemplo ilustra cómo la programación reactiva puede ser utilizada para el procesamiento de logs en tiempo real, transformando un flujo de logs brutos en un sistema de alertas eficiente.

```
val fuente = Source(logs)

val flujo = fuente
    .filter(log => log.contains("ERROR"))
    .map(log => Alerta(log))
    .to(Sink.foreach(alerta => enviarAlerta(alerta)))

flujo.run()
```



val fuente = Source(logs)

Crea un **flujo de logs** continuo a partir de una fuente de datos, estableciendo la base para el procesamiento reactivo.



.filter(...)

Aplica un filtro para identificar y propagar únicamente aquellos logs que contienen la palabra "ERROR", reduciendo el ruido.



.map(...)

Transforma cada log de error filtrado en un objeto **Alerta**, estructurando la información para su posterior gestión.



.to(Sink.foreach(...))

Define la **acción final**: para cada alerta generada, se invoca una función externa **enviarAlerta**, notificando sobre el error.

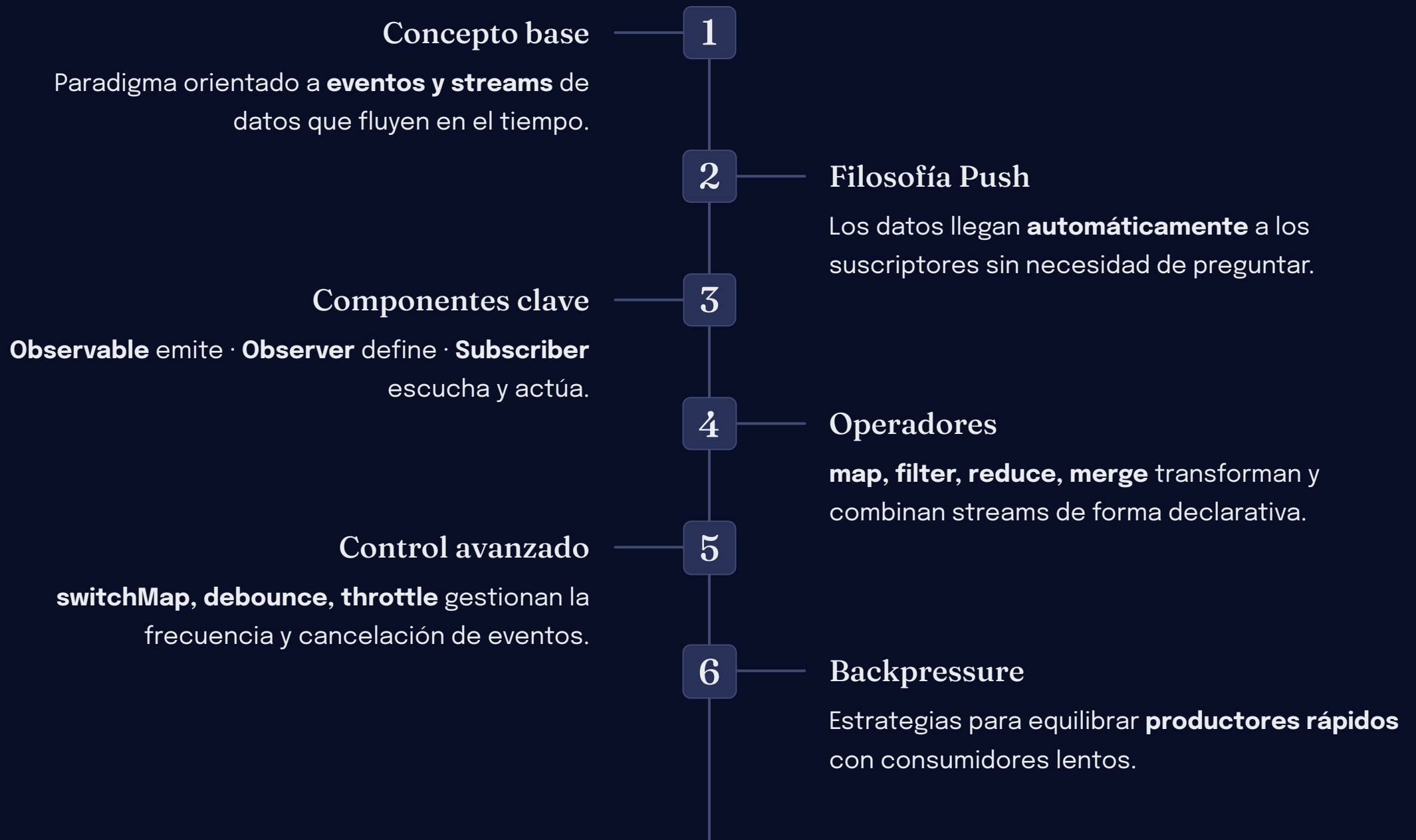


flujo.run()

Inicia la **ejecución del flujo** reactivo, permitiendo que los datos fluyan, se transformen y se actúe sobre ellos de forma asíncrona.

  **Conclusión:** Este enfoque permite reaccionar a eventos críticos de forma inmediata, mejorando la monitorización y la capacidad de respuesta ante incidencias, con un código conciso y declarativo.

Resumen: Programación Reactiva



✓ La Programación Reactiva es el paradigma que impulsa aplicaciones modernas como **Netflix, Uber, Spotify** y sistemas IoT en tiempo real.

Referencias

- <https://netflixtechblog.com/reactive-programming-in-the-netflix-api-with-rxjava-7811c3a1496a>
- <https://discord.com/blog/how-discord-scaled-elixir-to-5-000-000-concurrent-users>
- <https://www.uber.com/blog/new-rider-app-architecture/>
- <https://medium.com/airbnb-engineering/building-a-reactive-application-architecture-with-mvr-x-55416b9b32c8>
- <https://slack.engineering/flannel-an-application-level-edge-cache-to-make-slack-scale/>